

Abstract

Raiden, a production-deployed interprocess-communication (IPC) framework that provides robust shared-state management for large, long-lived, multi-process desktop applications. Raiden is built on top of **boost::interprocess** but layers onto it the structure that raw IPC primitives deliberately omit: it organizes all shared resources around a session, exposes hierarchical namespaces that mirror application structure, and gives every shared object an explicit, reference-counted lifecycle with automatic reclamation after abnormal termination.

Concretely, Raiden provides: session-scoped shared memory segments; typed, namespace-held entities (shared memory, message queues, mutexes/condition variables, interprocess caches, growable STL-like containers, and growable byte buffers); a cross-process procedure-call mechanism; and fail-safe cleanup of resources left behind by crashed processes.

The Problem

Split a desktop app into cooperating processes and C++ stops helping you at the boundary. Raw OS IPC primitives are powerful but intentionally minimal, leaving the hard problems to you:

Pointers don't cross. There is no shared heap; `shared_ptr` and raw pointers are meaningless in another process.

Stale, orphaned resources. Segments and named mutexes are kernel/filesystem-persistent and removed only by explicit calls, so a process that dies leaks them; they pile up into name collisions, resource exhaustion, and unsafe reuse.

Global names collide. Flat, globally visible names are shared across runs and even users, so unrelated sessions interfere by accident.

Locks are fragile under failure. A holder that dies inside a critical section leaves a named mutex locked forever, hanging every peer.

Ownership is implicit. No process clearly owns a resource's lifecycle; scattered create/destroy logic makes safe cleanup guesswork.

Shared segments can't grow. A segment's size is fixed once mapped, so growing a shared container means hand-rolled pre-allocation, remapping, or serialization.

Why Raiden?

Raiden is not a new IPC transport. It uses the same OS primitives as Boost and adds the architecture around them:

Crash-safe cleanup. Raiden reference-counts every entity and auto-reclaims it using the journal and "PID + start-time" liveness.

Collision-free naming. Raiden bakes the session identifier into every OS name, and groups entities in nested, session-scoped namespaces (e.g. `/documents/doc1`) that mirror application structure.

Explicit ownership. Raiden makes exclusive/shared ownership explicit, and with reference counting, the last live owner frees the object.

Discovery. Raiden's entities are self-describing and held in a templated registry, so a joining process finds them by name or kind and maps only what it needs, with no out-of-band name passing.

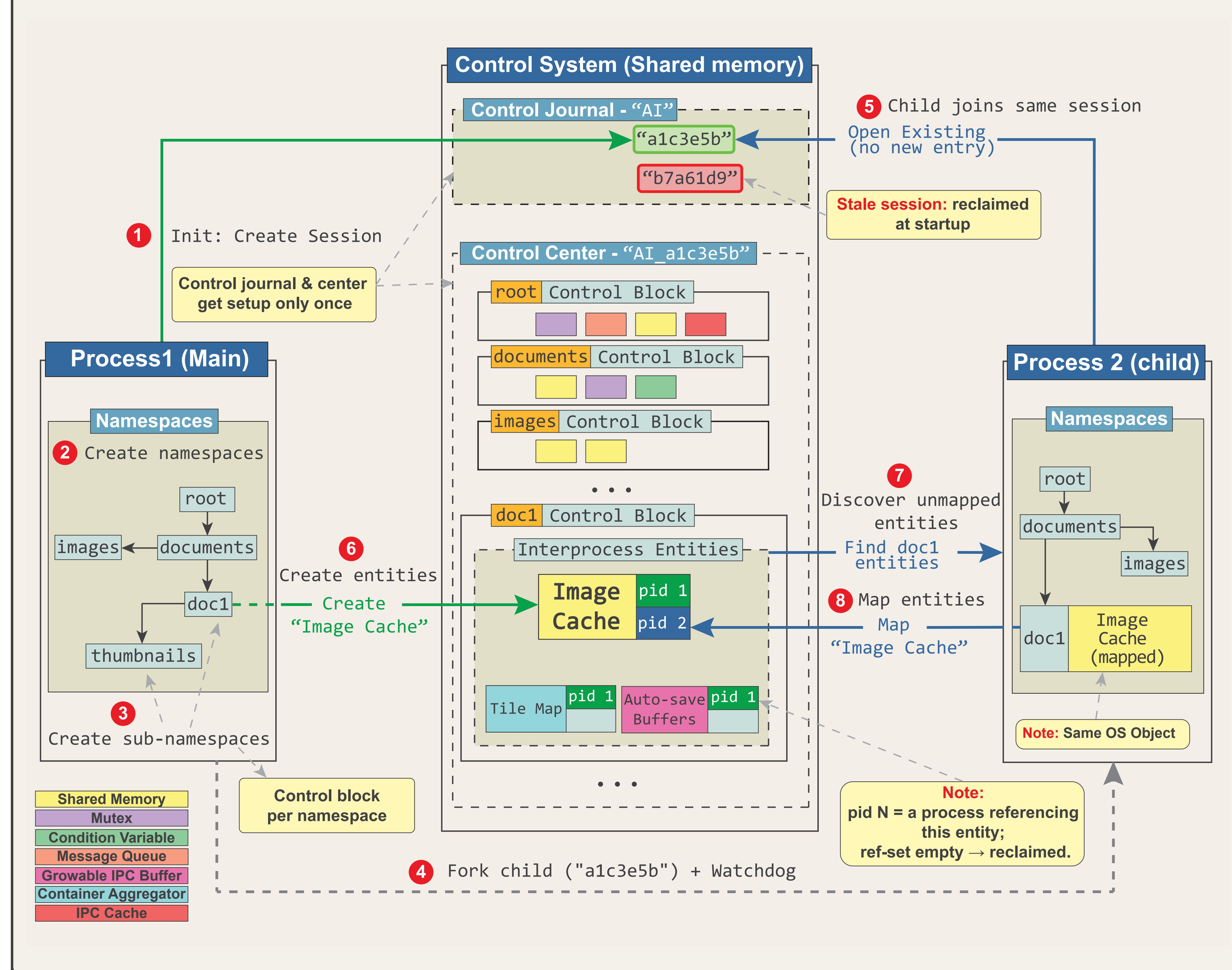
Managed locks. Raiden's `Sync<T>` confines the locks to tiny callback-free critical sections, and an orphaned lock is removed at startup to recover OS resources.

RAII. Allocations return RAII scope guards; `offset_ptr` keeps references valid in every process's address space.

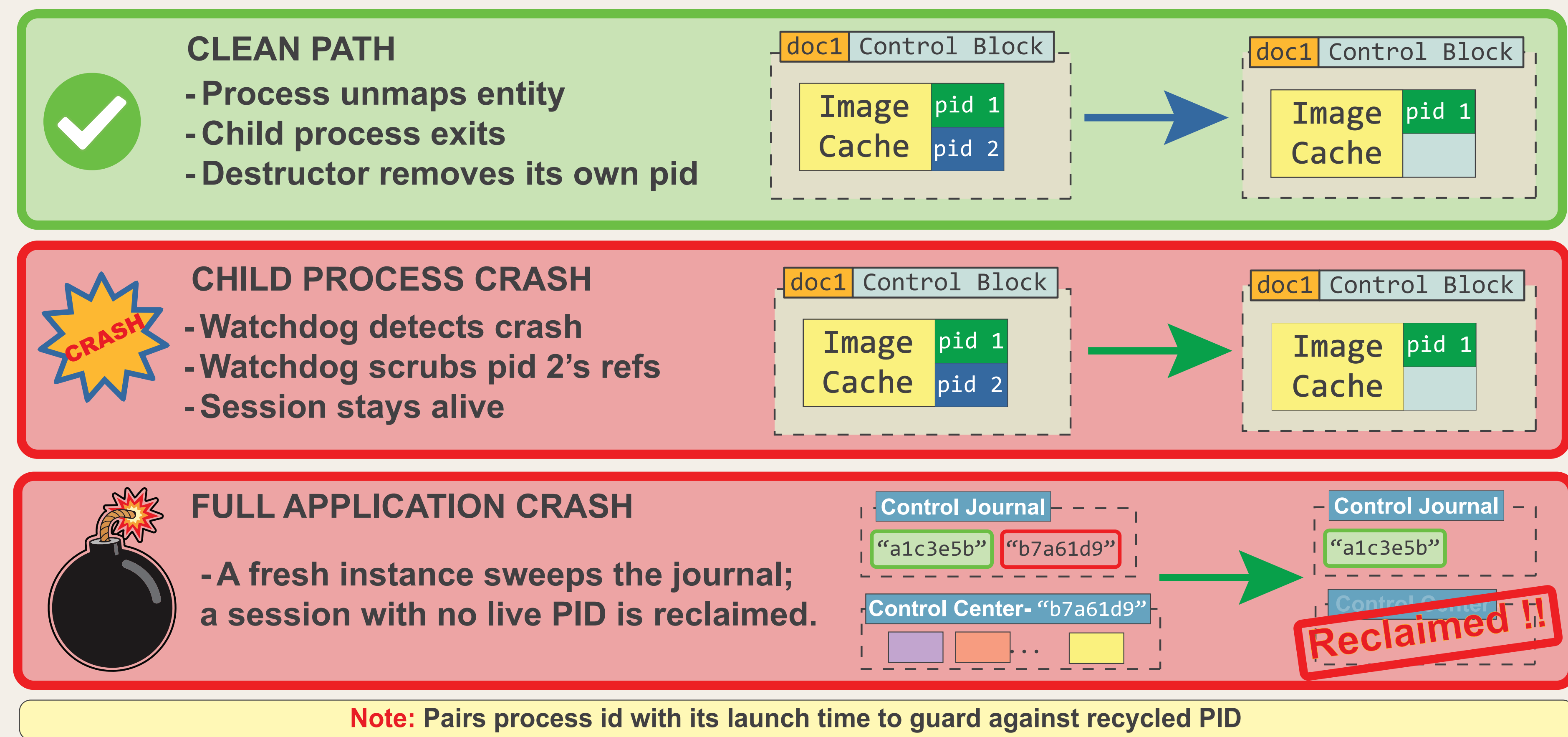
Growable shared data. `ContainerAggregator` spans one STL container over many segments, and `GrowableInterProcessBuffer` is an append-only stream, growing without remapping.

Also: Entity discovery by name or kind (no out-of-band names), type-safe cross-process RPC (variadic templates + `index_sequence`), and a policy-based cache (LRU / MRU / LFU / MFU).

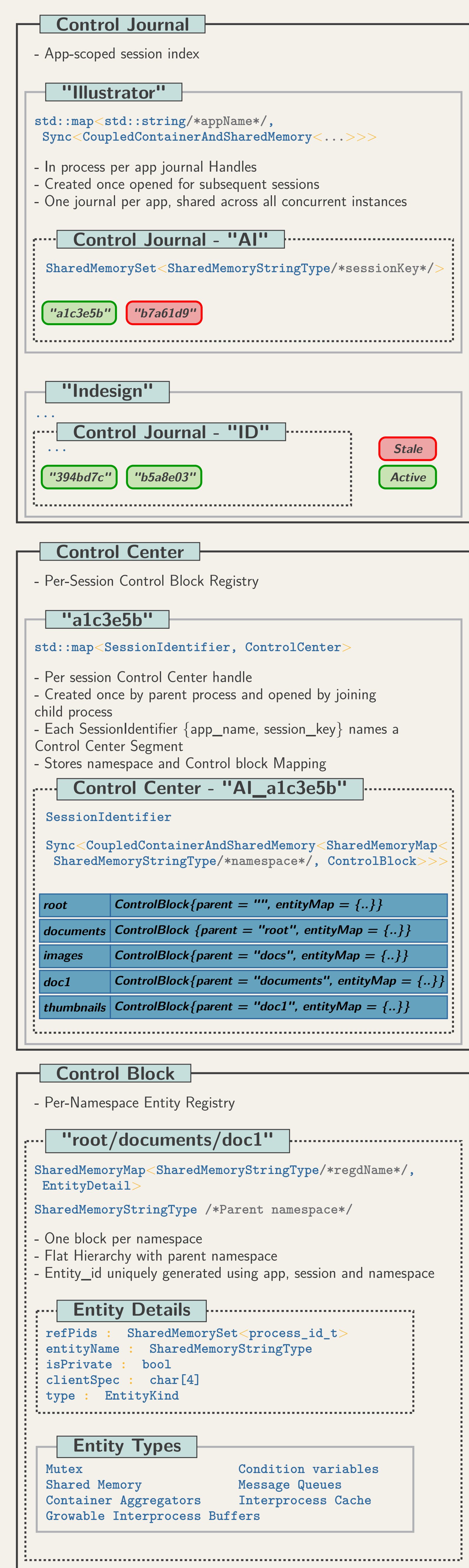
System Architecture: Shared Memory & Interprocess Discovery



Fail-Safe Cleanup & Stale-Session Reclamation



Internal Components



Completion Status

Completed and in production since Jan 2025. Raiden is fully implemented and shipping inside Adobe Illustrator, a commercial desktop application exceeding 50 million lines of code, and has been exercised across more than 2.4 million production sessions. Illustrator used Raiden to make its transition to a multi-process architecture both robust and fast and presently it underpins several multi-process features:

- **Shared imaging and asset caches** built on the interprocess cache and shared-memory entities, enabling zero-copy re-use of decoded images and assets across co-operating processes.
- **Auto file save, export, and document crash recovery:** document modification state is periodically shared over a growable interprocess buffer, including on-disk serialization/restore of large document state.
- **Render/tile pipelines** built on container aggregators and shared memory for dynamically growing shared datasets.

Raiden runs on macOS and Windows in production, with experimental support for additional platforms.

Benchmarks

The tables below summarize results from controlled benchmarks, where Table 1 and 2 report mean per-operation latency (1000 iterations; 100 for cross-process open) in microseconds; Table 3 and 4 report container aggregator benchmarks with CPU time over 100K operations in milliseconds.

Table 1: Framework overhead – Raiden-only (µs)					Table 2: Raiden vs. Boost – primitive create/open (µs)				
Operation	Avg (µs)				Operation	Boost	Raiden	Ovh.	Gain
Init: Create Journal + Center + ROOT namespace (one-time)	1932.30				Create	267.8	383.0	+115.2	-
Namespace creation (flat)	1.39				Open (diff. proc.)	221.9	246.4	+24.5	-
Namespace creation (chained)	1.61				Open (same proc.)	51.3	3.3	-	15.5×
Namespace creation (single)	29.45				Create	197.6	202.1	+4.5	-
Namespace creation (random depth(max 10))	1.47				Open (diff. proc.)	278.9	193.5	-85.4	-
Known session lookup	52.02				Open (same proc.)	47.9	3.4	-	14.0×
Unknown/Stale session lookup	12.71								

Table 3: Sequential aggregators vs. Boost (100K ops; ms)				Table 4: Associative aggregators vs. Boost (100K ops; ms)			
Operation	Boost	Raiden	Ovh.	Operation	Boost	Raiden	Ovh.
Vector				Set			
push_back	0.215	0.216	+0.5%	Insert	20.997	21.588	+2.8%
Random access	0.056	0.097	+73.2%	Find	14.288	14.485	+1.4%
Erase begin (worst)	354.22	354.89	+0.2%	Delete	22.208	22.538	+1.5%
List				Map			
push_back	3.447	3.579	+3.8%	Insert	21.116	21.874	+3.6%
Iterate	0.447	0.450	+0.7%	Find	14.604	14.767	+1.1%
pop_front	2.273	2.326	+2.3%	Delete	23.060	23.724	+2.9%

Namespace variants: *single* = first created after init; *flat* = direct children of root; *chained* = worst case, each nested under the previous. *Overhead* (Ovh.) is Raiden–Boost for Table 2 (µs) and (Raiden–Boost)/Boost for Tables 3-4 (%). *Gain* is the same-process re-open speedup once Raiden's cache returns the already-mapped object instead of re-opening it. Tables 3-4 use one 4 MB segment; growth across segments is not benchmarked, since a program can size its initial segments to the working set. The one large relative overhead is vector random access (+73.2%): a Boost vector is contiguous, so `v[i]` is a single address calculation, while Raiden's first finds which segment holds `i`. Each access is still near 1 ns (0.97 vs. 0.56 ns), negligible in practice.

C++17 is Illustrator's shipping production baseline, not a design limit. Raiden's contribution is architectural, so the patterns port unchanged to newer standards and stay relevant regardless of language version.

